SYSTEM DESIGN

# Service Mesh Visual Guide

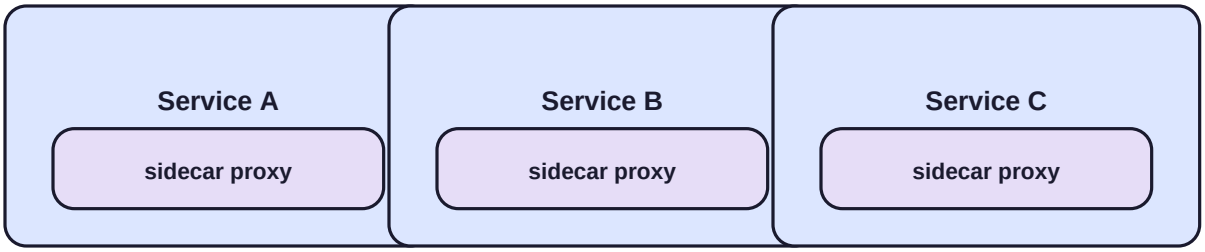
Networking as infrastructure:
sidecars, mTLS, and traffic policy without code changes.

**ANIMATED WALKTHROUGH**
Step through pages to play the animation

STEP 1/6

# The Sidecar Pattern

Every service instance gets a proxy roommate that owns its network.



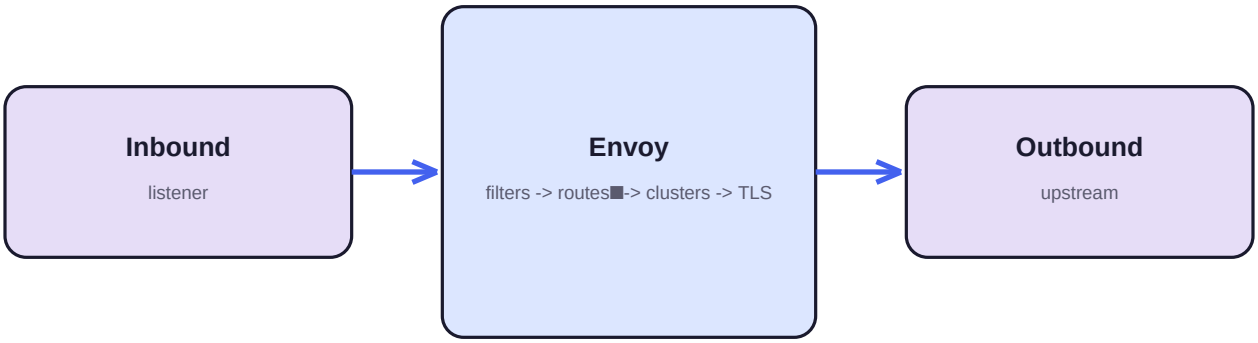

App code talks to localhost; the sidecar talks to the world. Zero library changes.

- Retries, timeouts, TLS, metrics -- all implemented once in the proxy, applied to every service.
- Cost: an extra hop per call (small latency), plus proxy CPU/memory per pod.

STEP 2/6

# Envoy: The Data Plane

The proxy that actually moves bytes (Istio, Linkerd's linkerd2-proxy, and friends).

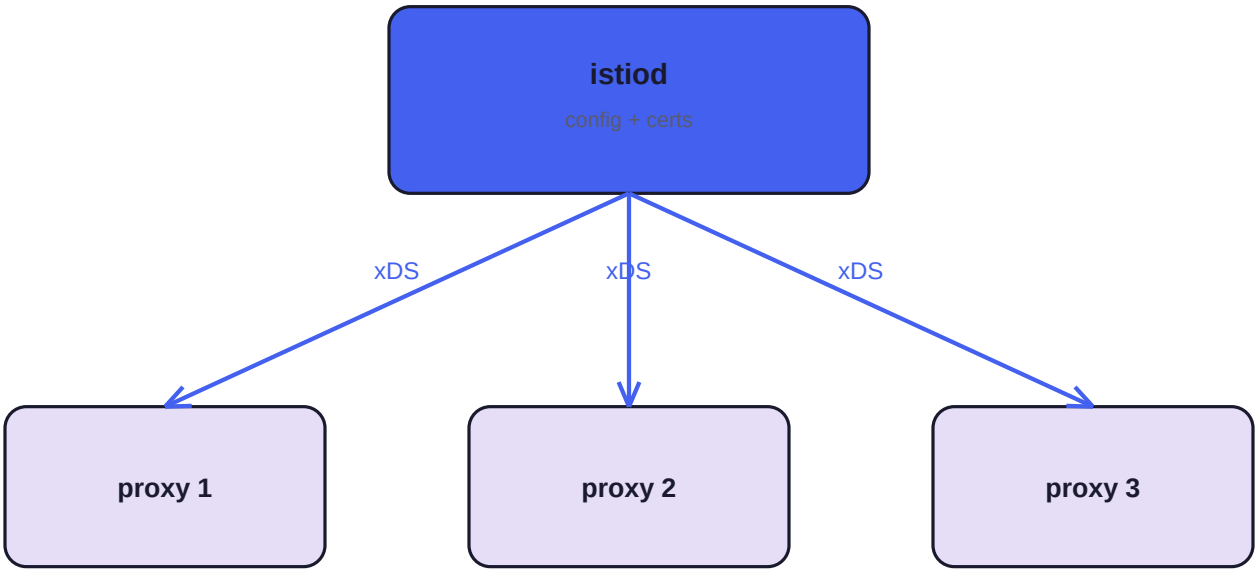



- Filter chain: auth, rate limit, transform -- then route by path/header to a cluster of endpoints.
- Outlier detection: Envoy ejects sick hosts automatically (passive circuit breaking).
- Hot-reloadable config: the control plane pushes new routes without dropping connections.

STEP 3/6

# Istio: The Control Plane

The brain that programs ten thousand sidecars as one.




- xDS APIs stream config: listeners, routes, clusters, endpoints -- eventually consistent, fast.
- Citadel (now in istiod): issues and rotates workload certificates for mTLS.
- You declare INTENT (VirtualService, DestinationRule); the mesh compiles it to proxy config.

STEP 4/6

# mTLS: Zero-Trust Between Services

Every hop authenticated and encrypted -- identity from the platform, not passwords.

- Both sides present certificates: no more 'internal network = trusted' thinking.
- AuthorizationPolicy: allow payments -> db ONLY from checkout -- enforced in the proxy.
- Permissive mode lets you migrate: plaintext today, strict mTLS tomorrow, per namespace.

STEP 5/6

# Traffic Policies Without Deploys

Shift, split, and protect traffic by editing config.

### Canary

v1 90% / v2 10% -- promote by weight, not by deploy

### Retries + timeouts

retry 2x on 5xx, 3s timeout -- per route, in the mesh

### Circuit breaking

max 100 concurrent to payments -- shed load before it cascades

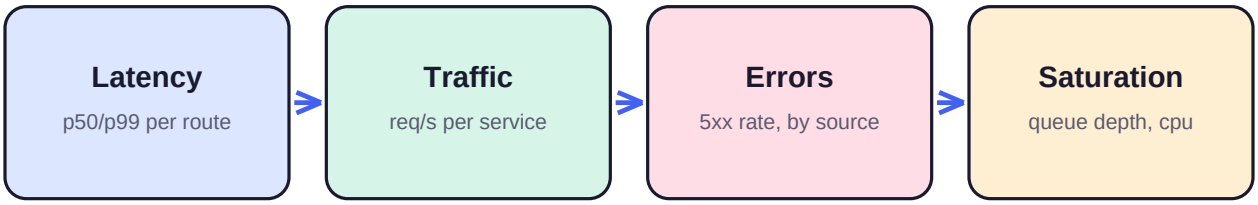
### Fault injection

delay 10% of calls 2s -- chaos drills in production config

STEP 6/6

# Telemetry for Free

Every proxy emits the golden signals -- no instrumentation needed.




- Distributed tracing: the proxy propagates trace headers even if the app ignores them.
- Service graph: who calls whom, how often, how slowly -- generated from proxy telemetry.
- This is the mesh's killer app for many teams: observability without begging every team to instrument.

STEP ✓/✓

# Key Takeaways

## Service Mesh — Remember

- Sidecars own the network: app talks to localhost, the proxy handles retries, TLS, metrics.
- Data plane (Envoy) moves bytes; control plane (istiod) programs all proxies via xDS.
- mTLS gives every workload a cryptographic identity -- zero-trust inside the cluster.
- Canary weights, retries, circuit breaking, and fault injection become config, not code.
- Uniform proxy telemetry yields tracing and service graphs without per-team instrumentation.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev